Predicting in Multi-Resolution Space, Not in Latent Space: Forecast-State Graph Resolution for Spatio-Temporal Prediction

Anonymous submission

Abstract

Most spatio-temporal forecasting models transform latent features through temporal and spatial modules and decode only a final future. This obscures how prediction evolves across temporal and graph resolutions. We instead formulate forecasting in configurable multi-resolution target spaces. Each forecast state is indexed by a temporal resolution and a graph resolution, and is therefore an explicit prediction object rather than a hidden feature. Distance- and capacity-constrained graph resolution constructs physically plausible region-level prediction spaces from the underlying traffic graph. The model then refines the forecast by predicting residual components in these resolution spaces and lifting them back to full node-time coordinates. Intermediate supervision is scale-matched: every state is compared only with the projection of the observable target into the same resolution space. Instead of optimizing a weighted sum of auxiliary losses, we use final-primary gradient projection, so that intermediate supervision contributes only gradient components that do not oppose the final prediction objective. Both temporal and graph resolution depths are configurable and selected experimentally.

Introduction

Spatio-temporal traffic forecasting maps graph-structured observations to future network states. It is a central problem in intelligent transportation systems, with direct relevance to congestion management, network control, and incident response. Recent forecasting models differ mainly in how they couple temporal dynamics and spatial dependence, but their dominant computational pattern is still to transform history through latent representations and decode the final forecast only at the end.

Separated spatio-temporal models typically assign temporal and spatial modules distinct roles and compose them serially, alternately, or in parallel. Typical temporal modules include recurrent units, temporal convolutions, decomposition blocks, frequency-domain operators, and more recent state-space models. Typical spatial modules include graph convolution (Defferrard, Bresson, and Vandergheynst 2016), adaptive graph learning as in Graph WaveNet (Wu et al. 2019), dynamic graph construction, and spatial attention. Joint spatio-temporal models instead build space-time graphs, tokenize node-time pairs, or mix temporal and spatial aggregation inside unified layers. These two families differ in coupling style and complexity, but both usually operate through hidden feature transformations.

Direct history-to-future prediction is therefore a strong baseline rather than a strawman. It is often stable and efficient, but it collapses coarse and fine future structure into one final mapping. Latent multi-scale modules partially address this limitation, yet they still do not guarantee that intermediate variables correspond to interpretable forecasts. Even when a model contains multiple temporal or spatial scales, the propagated objects are usually hidden features whose meanings are not fixed in prediction coordinates.

A natural alternative is to predict directly in target space at multiple resolutions. For spatio-temporal forecasting, however, multi-resolution prediction should not be restricted to temporal scales only. Graph resolution is equally important: a forecast may be coarse in time, coarse in graph structure, or coarse in both. This suggests a unified prediction-space view in which temporal resolution and graph resolution are two coordinates of the same intermediate forecast space rather than two disjoint modules.

We therefore define a configurable sequence of multi-resolution prediction spaces. Each forecast state is indexed by a temporal length and a graph cluster capacity, and each state is supervised only in its own prediction space. The model refines the forecast by predicting residual components in these spaces and lifting them back to full node-time coordinates. Every intermediate object is thus an explicit forecast in observable coordinates rather than a latent feature.

This target-space formulation creates a second challenge: naive stage-wise optimization may improve intermediate states while harming the final forecast, and weighted-sum auxiliary losses introduce manual loss balancing while allowing auxiliary gradients to conflict with the final objective. We therefore use final-primary gradient projection: the final prediction loss defines the primary descent direction, and intermediate resolution supervision is allowed to update parameters only through gradient components that do not oppose it.

The resulting framework organizes forecasting as refinement across configurable multi-resolution prediction spaces

indexed by temporal length and graph capacity. Distance- and capacity-constrained graph resolution ensures that coarse graph prediction spaces remain physically plausible, while the resolution schedule itself is selected experimentally and may refine temporal resolution, graph resolution, or both at different stages.

Our contributions are threefold:

- **Multi-resolution target-space forecasting.** We formulate spatio-temporal prediction as a sequence of forecast states in configurable prediction spaces indexed by temporal resolution and graph resolution.
- **Distance- and capacity-constrained graph-resolution spaces.** We construct graph-resolution prediction spaces with capacity-constrained, distance-aware node-to-region assignments, and perform residual forecast refinement through mixed structural-adaptive graph operators.
- **Final-primary optimization for intermediate supervision.** We supervise intermediate resolution states with scale-matched targets, but replace weighted-sum auxiliary training with gradient projection against the final prediction objective.

Methodology

We model multi-step forecasting as residual refinement in a configurable sequence of multi-resolution prediction spaces. Each stage $s \in \{1, \dots, S\}$ is associated with a resolution state $\omega_s = (h_s, K_s)$, where h_s specifies a temporal resolution and K_s specifies a graph cluster capacity. The model predicts in the corresponding prediction space, lifts the prediction back to full node-time coordinates, and accumulates it into the current forecast. Temporal and graph resolution are therefore two axes of the same prediction space rather than two disjoint modules.

Problem Formulation

Let $\mathbf{X} \in \mathbb{R}^{B\times P\times N\times C_x}$ denote the observed history, $\mathbf{Y} \in \mathbb{R}^{B\times H\times N\times C_y}$ the future target, and $\widehat{\mathbf{Y}} \in \mathbb{R}^{B\times H\times N\times C_y}$ the final prediction. Here B is the batch size, P the input length, H the forecasting horizon, N the number of nodes, and C_x and C_y the input and target channel dimensions. The full target space is

$$\mathcal{Y} = \mathbb{R}^{B\times H\times N\times C_y}. \tag{1}$$

Multi-Resolution Prediction Spaces

Let

$$\begin{aligned} \Omega &= \{\omega_s\}_{s=1}^{S} = \{(h_s, K_s)\}_{s=1}^{S}, \\ h_1 &\le h_2 \le \cdots \le h_S = H, \\ K_1 &\ge K_2 \ge \cdots \ge K_S = 1, \end{aligned} \tag{2}$$

be the resolution schedule. The number of stages S is not fixed; it is a configurable architectural hyperparameter selected by experiment. Some stages may refine only temporal resolution, some only graph resolution, and others both simultaneously.

For $\omega_s = (h_s, K_s)$, define

$$M_s = \left\lceil \frac{N}{K_s} \right\rceil, \qquad \mathcal{Y}_{\omega_s} = \mathbb{R}^{B\times h_s\times M_s\times C_y}. \tag{3}$$

Each $\mathcal{Y}_{\omega_s}$ is not a hidden feature space, but a lower-resolution prediction space obtained by reducing the temporal axis and/or the graph axis of the observable target.

The temporal projection operator is

$$\mathcal{D}_{h_s} : \mathcal{Y} \to \mathbb{R}^{B\times h_s\times N\times C_y}, \tag{4}$$

and, when H is divisible by h_s,

$$\left(\mathcal{D}_{h_s}(\mathbf{Y})\right)_\tau = \frac{1}{a_s} \sum_{t=(\tau-1)a_s+1}^{\tau a_s} \mathbf{Y}_t, \qquad a_s = \frac{H}{h_s}. \tag{5}$$

Otherwise we use adaptive average pooling on the temporal axis. The temporal lifting operator is

$$\mathcal{U}_{h_s} : \mathbb{R}^{B\times h_s\times N\times C_y} \to \mathcal{Y}, \tag{6}$$

implemented by interpolation or block repetition.

Using the stagewise assignment matrices defined in the next subsection, let

$$\begin{aligned} &\mathbf{C}_s \in \{0,1\}^{N\times M_s}, \\ &\mathbf{D}_s = \operatorname{diag}\!\left(\mathbf{C}_s^\top \mathbf{1}\right), \\ &\mathbf{P}_s = \mathbf{D}_s^{-1}\mathbf{C}_s^\top. \end{aligned} \tag{7}$$

Graph projection and lifting act on the node axis:

$$\begin{aligned} \mathcal{P}_{K_s}(\mathbf{Y}) &= \mathbf{P}_s\mathbf{Y}, \\ \mathcal{L}_{K_s}(\mathbf{Z}) &= \mathbf{C}_s\mathbf{Z}. \end{aligned} \tag{8}$$

We then define the unified projection and lifting operators

$$\begin{aligned} \Pi_s &= \mathcal{P}_{K_s} \circ \mathcal{D}_{h_s} : \mathcal{Y} \to \mathcal{Y}_{\omega_s}, \\ \Lambda_s &= \mathcal{U}_{h_s} \circ \mathcal{L}_{K_s} : \mathcal{Y}_{\omega_s} \to \mathcal{Y}. \end{aligned} \tag{9}$$

When $h_s = H$, the temporal operators reduce to identities on the temporal axis. When $K_s = 1$, the graph operators reduce to identities on the node axis.

Distance- and Capacity-Constrained Graph Resolution

Let $\mathbf{A} \in \mathbb{R}^{N\times N}$ denote the raw adjacency matrix and define

$$\mathbf{A}^{\mathrm{sym}} = \frac{1}{2}\left(\mathbf{A} + \mathbf{A}^\top\right). \tag{10}$$

Let d_{ij} be the physical distance between nodes i and j. With $Q_{0.95}(d)$ denoting the 95-th percentile of pairwise distances, the normalized distance is

$$\widetilde{d}_{ij} = \operatorname{clip}\!\left(\frac{d_{ij}}{Q_{0.95}(d)}, 0, 1\right). \tag{11}$$

The distance-aware affinity is

$$W_{ij} = A_{ij}^{\mathrm{sym}} \exp\!\left(-\frac{\widetilde{d}_{ij}^2}{\sigma_d^2}\right). \tag{12}$$

For each stage with $K_s > 1$, we construct a capacity-constrained cluster assignment. Define

$$\begin{aligned} \bar{\mathbf{W}} &= \mathbf{W} + \mathbf{I}, \\ (\mathbf{D}_g)_{ii} &= \sum_j \bar{W}_{ij}, \\ \mathbf{S}_g &= \mathbf{D}_g^{-1/2}\bar{\mathbf{W}}\mathbf{D}_g^{-1/2}, \end{aligned} \tag{13}$$

and let

$$\mathbf{Z}_g = \left[\mathbf{u}_1, \ldots, \mathbf{u}_{d_g}\right] \in \mathbb{R}^{N \times d_g}. \tag{14}$$

With $\mathbf{z}_i$ denoting the i-th row of $\mathbf{Z}_g$, we solve

$$\mathbf{C}_s = \arg\min_{C \in \mathcal{C}(K_s),\, \{\boldsymbol{\mu}_q\}} \sum_{i=1}^{N} \sum_{q=1}^{M_s} C_{iq} \left(\|\mathbf{z}_i - \boldsymbol{\mu}_q\|_2^2 + \lambda_d \bar{d}_{iq} \right), \tag{15}$$

where

$$\mathcal{C}(K_s) = \left\{ C \in \{0,1\}^{N \times M_s} : \begin{array}{r} C\mathbf{1} = \mathbf{1}, \\ 1 \le C_{:,q}^{\top}\mathbf{1} \le K_s \end{array} \right\}. \tag{16}$$

Here $\bar{d}_{iq}$ denotes the average normalized distance from node i to nodes assigned to cluster q. If $K_s = 1$, we set

$$\begin{aligned} \mathbf{C}_s &= \mathbf{I}_N, \\ \mathbf{P}_s &= \mathbf{I}_N. \end{aligned} \tag{17}$$

The resulting graph resolution is part of the stagewise prediction space $\mathcal{Y}_{\omega_s}$ rather than a separate pooling head.

Forecast-State Refinement

For each resolution state ω_s, we construct a resolution-specific graph operator. The structural graph is

$$\mathbf{A}_s^{\mathrm{str}} = \mathrm{RowNorm}(\mathbf{P}_s \mathbf{W} \mathbf{C}_s + \mathbf{I}). \tag{18}$$

Let

$$\mathbf{E}_s^{\mathrm{src}}, \mathbf{E}_s^{\mathrm{dst}} \in \mathbb{R}^{M_s \times d} \tag{19}$$

be learnable embeddings and define

$$\mathbf{Q}_s = \mathbf{E}_s^{\mathrm{src}} \left(\mathbf{E}_s^{\mathrm{dst}}\right)^{\top}. \tag{20}$$

With stagewise top-k budget k_s,

$$\bar{Q}_{s,ij} = \begin{cases} Q_{s,ij}, & j \in \mathrm{TopK}(Q_{s,i:}, k_s), \\ -\infty, & \text{otherwise}, \end{cases} \tag{21}$$

and the mixed graph operator is

$$\begin{aligned} \mathbf{A}_s^{\mathrm{adp}} &= \mathrm{Softmax}\left(\frac{\bar{\mathbf{Q}}_s}{\tau}\right), \\ \mathbf{A}_s &= (1 - \lambda_s)\, \mathbf{A}_s^{\mathrm{adp}} + \lambda_s \mathbf{A}_s^{\mathrm{str}}. \end{aligned} \tag{22}$$

We initialize the full-resolution forecast by

$$\widehat{\mathbf{Y}}^{(0)} = 0. \tag{23}$$

At stage s, the projected previous forecast is

$$\bar{\mathbf{Z}}_s = \Pi_s\left(\widehat{\mathbf{Y}}^{(s-1)}\right). \tag{24}$$

We then predict a residual forecast component in the corresponding multi-resolution prediction space:

$$\mathbf{R}_s = F_s\left(\mathbf{X}, \bar{\mathbf{Z}}_s; \mathbf{A}_s\right) \in \mathcal{Y}_{\omega_s}. \tag{25}$$

Here F_s denotes the stagewise forecast-state refinement module. It may instantiate temporal, graph, or combined operations inside the same resolution space, so temporal and graph refinement remain part of a single prediction-space update.

The residual component is lifted back to the full target space by

$$\Delta\widehat{\mathbf{Y}}^{(s)} = \Lambda_s(\mathbf{R}_s), \tag{26}$$

and the full-resolution forecast is updated as

$$\widehat{\mathbf{Y}}^{(s)} = \widehat{\mathbf{Y}}^{(s-1)} + \alpha_s \Delta\widehat{\mathbf{Y}}^{(s)}. \tag{27}$$

The final prediction is

$$\widehat{\mathbf{Y}} = \widehat{\mathbf{Y}}^{(S)}. \tag{28}$$

Every intermediate object $\mathbf{R}_s$ is therefore predicted in $\mathcal{Y}_{\omega_s}$, a resolution-specific prediction space with explicit temporal and graph meaning, rather than in an unconstrained latent space.

Scale-Matched Supervision without Weighted-Sum Loss

For each stage s, the observable target in the same prediction space is

$$\mathbf{Y}_s^{\star} = \Pi_s(\mathbf{Y}). \tag{29}$$

The intermediate auxiliary objective is then

$$L_s^{\mathrm{aux}} = \ell\Big(\Pi_s\Big(\widehat{\mathbf{Y}}^{(s)}\Big), \Pi_s(\mathbf{Y})\Big), \qquad s = 1, \ldots, S-1. \tag{30}$$

The final prediction loss is

$$L_0 = \ell\Big(\widehat{\mathbf{Y}}^{(S)}, \mathbf{Y}\Big), \tag{31}$$

and the auxiliary set is

$$\mathcal{A} = \{L_s^{\mathrm{aux}} : s = 1, \ldots, S-1\}. \tag{32}$$

We do not optimize a weighted sum of these objectives. Temporal and graph supervision are unified as supervision in the corresponding multi-resolution prediction spaces.

Final-Primary Gradient Projection

Let θ denote all trainable parameters, flattened only for inner products and norms. The final-loss gradient is

$$g_0 = \nabla_\theta L_0. \tag{33}$$

For each auxiliary objective $L_s^{\mathrm{aux}} \in \mathcal{A}$, define

$$g_s = \nabla_\theta L_s^{\mathrm{aux}}. \tag{34}$$

Its projected version is

$$\widetilde{g}_s = g_s - \min\left(0, \frac{g_s^{\top} g_0}{\|g_0\|_2^2 + \epsilon}\right) g_0. \tag{35}$$

This compactly removes only the component of g_s that conflicts with g_0, so that $\widetilde{g}_s^{\top} g_0 \ge 0$. For $S > 1$, the aggregated auxiliary gradient is

$$g_{\mathrm{aux}} = \frac{1}{S-1} \sum_{s=1}^{S-1} \widetilde{g}_s, \tag{36}$$

followed by the norm cap

$$g_{\mathrm{aux}} \leftarrow g_{\mathrm{aux}} \cdot \min\left(1, \frac{\rho \|g_0\|_2}{\|g_{\mathrm{aux}}\|_2 + \epsilon}\right). \tag{37}$$

Algorithm 1: Forecast-State Refinement in Multi-Resolution Prediction Spaces

Require: History $\mathbf{X}$, target $\mathbf{Y}$, resolution schedule $\Omega = \{(h_s, K_s)\}_{s=1}^{S}$, graph hyperparameters, top-k values

Construct the distance-aware affinity $\mathbf{W}$ from $\mathbf{A}^{\mathrm{sym}}$ and the normalized distances
for $s = 1$ to S **do**
  Construct $\mathbf{C}_s$, $\mathbf{P}_s$, and $\mathbf{A}_s$
end for
Initialize $\widehat{\mathbf{Y}}^{(0)} \leftarrow 0$
for $s = 1$ to S **do**
  $\bar{\mathbf{Z}}_s \leftarrow \Pi_s(\widehat{\mathbf{Y}}^{(s-1)})$
  $\mathbf{R}_s \leftarrow F_s(\mathbf{X}, \bar{\mathbf{Z}}_s; \mathbf{A}_s)$
  $\widehat{\mathbf{Y}}^{(s)} \leftarrow \widehat{\mathbf{Y}}^{(s-1)} + \alpha_s \Lambda_s(\mathbf{R}_s)$
end for
Compute $L_0 = \ell(\widehat{\mathbf{Y}}^{(S)}, \mathbf{Y})$
for $s = 1$ to $S - 1$ **do**
  $L_s^{\mathrm{aux}} \leftarrow \ell(\Pi_s(\widehat{\mathbf{Y}}^{(s)}), \Pi_s(\mathbf{Y}))$
end for
Compute $g_0 = \nabla_\theta L_0$
for $s = 1$ to $S - 1$ **do**
  Compute $g_s = \nabla_\theta L_s^{\mathrm{aux}}$
  Project g_s against g_0 if conflicting
end for
Average the projected auxiliary gradients and apply the norm cap
Update parameters with $g = g_0 + g_{\mathrm{aux}}$

Ensure: $\widehat{\mathbf{Y}} = \widehat{\mathbf{Y}}^{(S)}$

The final update gradient is

$$g = g_0 + g_{\mathrm{aux}}, \tag{38}$$

and the parameter update is

$$\theta \leftarrow \theta - \eta g. \tag{39}$$

This is not a weighted-sum objective. The final loss defines the primary descent direction, and intermediate resolution supervision can contribute only directions that do not oppose it.

Algorithm and Complexity

The critical path depends on the number of resolution states S, because forecast refinement is sequential in the schedule Ω. This does not impose a fixed hierarchy: the schedule is configurable and selected experimentally. Some schedules refine only temporal resolution for several stages, some only graph resolution, and others interleave both. Stage-wise assignments $\mathbf{C}_s$, projection matrices $\mathbf{P}_s$, and structural graphs $\mathbf{A}_s^{\mathrm{str}}$ depend only on the dataset graph and the chosen schedule, so they can be precomputed or cached. Training-time overhead is dominated by the auxiliary gradient computations for stages $s < S$. This overhead grows with the number of supervised resolution states, but it avoids manual loss-weight tuning and preserves the final prediction objective.

References

Bober, J.; Monod, A.; Saucan, E.; and Webster, K. N. 2022. Rewiring Networks for Graph Neural Network Training Using Discrete Geometry. *arXiv preprint arXiv:2207.08026*.

Bodnar, C.; Di Giovanni, F.; Chamberlain, B. P.; Liò, P.; and Bronstein, M. M. 2022. Neural Sheaf Diffusion: A Topological Perspective on Heterophily and Oversmoothing in GNNs. *arXiv preprint arXiv:2202.04579*.

Caruso, A.; Venturin, J.; Giambagli, L.; Rolando, E.; Noé, F.; and Clementi, C. 2025. Extending the RANGE of Graph Neural Networks: Relaying Attention Nodes for Global Encoding. *arXiv preprint arXiv:2502.13797*.

Chen, J.; Zuo, H.; Wang, H. P.; Miao, S.; Li, P.; and Ying, R. 2025. Towards A Universal Graph Structural Encoder. *arXiv preprint arXiv:2504.10917*.

Defferrard, M.; Bresson, X.; and Vandergheynst, P. 2016. Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering. In *Advances in Neural Information Processing Systems*.

Feng, Y.; You, H.; Zhang, Z.; Ji, R.; and Gao, Y. 2018. Hypergraph Neural Networks. *arXiv preprint arXiv:1809.09401*.

Fu, X.; Zhang, B.; Dong, Y.; Chen, C.; and Li, J. 2022. Federated Graph Machine Learning: A Survey of Concepts, Techniques, and Applications. *arXiv preprint arXiv:2207.11812*.

Gao, H.; Han, X.; Huang, J.; Wang, J.-X.; and Liu, L.-P. 2022. PatchGT: Transformer over Non-trainable Clusters for Learning Graph Representations. *arXiv preprint arXiv:2211.14425*.

Liu, J.; Yang, C.; Lu, Z.; Chen, J.; Li, Y.; Zhang, M.; Bai, T.; Fang, Y.; Sun, L.; Yu, P. S.; and Shi, C. 2023. Towards Graph Foundation Models: A Survey and Beyond. *arXiv preprint arXiv:2310.11829*.

Peng, C.; He, J.; and Xia, F. 2024. Learning on Multimodal Graphs: A Survey. *arXiv preprint arXiv:2402.05322*.

Rampášek, L.; Galkin, M.; Dwivedi, V. P.; Luu, A. T.; Wolf, G.; and Beaini, D. 2022. Recipe for a General, Powerful, Scalable Graph Transformer. In *Advances in Neural Information Processing Systems*.

Shirzad, H.; Velingker, A.; Venkatachalam, B.; Sutherland, D. J.; and Sinop, A. K. 2023. Exphormer: Sparse Transformers for Graphs. *arXiv preprint arXiv:2303.06147*.

Wu, Z.; Pan, S.; Long, G.; Jiang, J.; and Zhang, C. 2019. Graph WaveNet for Deep Spatial-Temporal Graph Modeling. *arXiv preprint arXiv:1906.00121*.

Ying, C.; Cai, T.; Luo, S.; Zheng, S.; Ke, G.; He, D.; Shen, Y.; and Liu, T.-Y. 2021. Do Transformers Really Perform Bad for Graph Representation? In *Advances in Neural Information Processing Systems*.

Ying, R.; You, J.; Morris, C.; Ren, X.; Hamilton, W. L.; and Leskovec, J. 2018. Hierarchical Graph Representation Learning with Differentiable Pooling. In *Advances in Neural Information Processing Systems*.

Zheng, X.; Wang, Y.; Liu, Y.; Li, M.; Zhang, M.; Jin, D.; Yu, P. S.; and Pan, S. 2022. Graph Neural Networks for Graphs with Heterophily: A Survey. *arXiv preprint arXiv:2202.07082*.

Implementation Notes and Evaluation Status

The architecture is controlled by a configurable resolution schedule $\Omega = \{\omega_s\}_{s=1}^{S} = \{(h_s, K_s)\}_{s=1}^{S}$. The number of stages S is not fixed by the method definition. Consecutive stages may refine only temporal resolution, only graph resolution, or both, depending on the schedule selected experimentally.

The stagewise assignments $\mathbf{C}_s$, projection matrices $\mathbf{P}_s$, and structural graphs $\mathbf{A}_s^{\mathrm{str}}$ depend only on the dataset graph and the chosen schedule. They can therefore be precomputed before training or cached per dataset. The remaining training-time overhead is dominated by the gradient-projection procedure, whose cost grows with the number of supervised intermediate resolution states.

At the time of this manuscript revision, only experimental results that match the method definition in the main text should be reported as evidence for the framework. Earlier results from mismatched training or graph-resolution variants should be treated only as implementation history rather than quantitative support.

Resolution-Space Forecast States and Optimization

This appendix clarifies why intermediate forecasts remain target-space prediction objects rather than latent features, how the unified projection and lifting operators define stagewise prediction spaces, why final-primary gradient projection protects the final objective in first order, and why distance/capacity graph resolution is not ordinary graph pooling.

Target-Space Forecast States vs. Latent Features

For each resolution state ω_s, the observable target in the same prediction space is

$$\mathbf{Y}_s^{\star} = \Pi_s(\mathbf{Y}). \tag{40}$$

The supervised object therefore lives in fixed target coordinates defined by temporal resolution h_s, graph resolution K_s, and output channels. This is different from a latent hidden state, whose coordinates may be freely changed as long as the final decoder changes accordingly.

Proposition 1 (Target-space anchoring of forecast states)**.** *Assume that the loss ℓ satisfies $\ell(\mathbf{u}, \mathbf{v}) = 0$ if and only if $\mathbf{u} = \mathbf{v}$. Then, for any $s \in \{1, \dots, S-1\}$,*

$$\begin{aligned} &\mathbb{E}\Big[\ell\Big(\Pi_s\Big(\widehat{\mathbf{Y}}^{(s)}\Big), \Pi_s(\mathbf{Y})\Big)\Big] = 0 \\ &\qquad \Longrightarrow \quad \Pi_s\Big(\widehat{\mathbf{Y}}^{(s)}\Big) = \Pi_s(\mathbf{Y}) \quad \textit{a.s.} \end{aligned} \tag{41}$$

Proof. The claim follows immediately from point separation of ℓ in the observable target coordinates. □

Proposition 1 does not claim global parameter identifiability. It states only that the supervised object is defined in target space rather than in an arbitrary hidden basis.

Latent Hidden States Remain Reparameterization-Dependent

Consider a latent multi-stage model with hidden variables $\mathbf{H}_l = e_l(\mathbf{X}, \mathbf{H}_{l-1})$ and final decoder $r(\mathbf{H}_L)$. For any bijection $\phi_l : \mathcal{H}_l \to \mathcal{H}_l$, define

$$\mathbf{H}_l' = \phi_l(\mathbf{H}_l), \qquad r'(\mathbf{H}_L') = r\big(\phi_L^{-1}(\mathbf{H}_L')\big). \tag{42}$$

The transformed model produces exactly the same observable prediction as the original one. Latent coordinates are therefore defined only up to reparameterization. By contrast, each forecast state $\Pi_s(\widehat{\mathbf{Y}}^{(s)})$ is tied to the observable target $\mathbf{Y}_s^{\star}$, so its semantics are fixed by target coordinates rather than by hidden-basis choice.

Resolution-Space Projection and Lifting

The stagewise prediction space is defined by the unified operators

$$\Pi_s = \mathcal{P}_{K_s} \circ \mathcal{D}_{h_s}, \qquad \Lambda_s = \mathcal{U}_{h_s} \circ \mathcal{L}_{K_s}. \tag{43}$$

When $h_s = H$, the temporal operators reduce to identities on the temporal axis; when $K_s = 1$, the graph operators reduce to identities on the node axis. The lifted quantity $\Lambda_s(\Pi_s(\mathbf{Y}))$ need not equal $\mathbf{Y}$. It is instead the best approximation to $\mathbf{Y}$ representable at the resolution specified by ω_s, and the refinement module learns residual corrections relative to that approximation.

This viewpoint clarifies the role of intermediate states. The projected forecast $\bar{\mathbf{Z}}_s = \Pi_s(\widehat{\mathbf{Y}}^{(s-1)})$ is not a latent summary of the previous stage; it is the current forecast expressed in the same target-space coordinates in which the next residual component will be predicted.

First-Order View of Final-Primary Gradient Projection

For each auxiliary objective, the projected gradient satisfies

$$\widetilde{g}_s^{\top} g_0 \geq 0. \tag{44}$$

If the overall update direction is $g = g_0 + g_{\mathrm{aux}}$, then for a sufficiently small learning rate η,

$$L_0(\theta - \eta g) = \; L_0(\theta) - \eta g_0^{\top} g + O(\eta^2). \tag{45}$$

Since every projected auxiliary component satisfies $\widetilde{g}_s^{\top} g_0 \geq 0$, the auxiliary term does not oppose the final descent direction in first order. This is the core distinction from weighted-sum auxiliary optimization: the final objective remains primary, and intermediate supervision can contribute only directions that are non-conflicting with it.

For completeness, the projected auxiliary update can be written in the same compact form as the main text:

$$\widetilde{g}_s = g_s - \min\left(0, \frac{g_s^{\top} g_0}{\|g_0\|_2^2 + \epsilon}\right) g_0. \tag{46}$$

Why Distance/Capacity Graph Resolution Is Not Ordinary Pooling

Ordinary graph pooling typically compresses node representations into a coarser graph and continues computation

there, often with the coarse representation itself becoming the main carrier of subsequent information. In our setting, the assignment $\mathbf{C}_s \in \{0,1\}^{N \times M_s}$ and projection $\mathbf{P}_s$ define a temporary graph-resolution prediction space:

$$\begin{aligned} \bar{\mathbf{Z}}_s &= \Pi_s\left(\widehat{\mathbf{Y}}^{(s-1)}\right), \\ \widehat{\mathbf{Y}}^{(s)} &= \widehat{\mathbf{Y}}^{(s-1)} + \alpha_s \Lambda_s(\mathbf{R}_s). \end{aligned} \tag{47}$$

The cluster-level quantity is therefore not a terminal coarse graph representation. It is an intermediate prediction space used to produce a residual correction that is lifted back to node-time coordinates. The purpose of distance and capacity constraints is likewise not generic graph compression; it is to define graph-resolution forecast spaces whose corrections remain recoverable in the original prediction coordinates.

Appendix: Compact Taxonomy of Graph Handling Methods

Graph-based spatio-temporal models differ not only in their message-passing rule, but also in how they define, compress, modify, or encode the underlying graph structure. Given a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with adjacency matrix $\mathbf{A}$, node features $\mathbf{X}$, node states $\mathbf{H}^{(\ell)}$, and optional positional or structural encodings $\mathbf{P}$, a generic graph update can be written as

$$\mathbf{H}^{(\ell+1)} = \mathcal{F}_\theta(\mathbf{H}^{(\ell)}, \mathbf{A}, \mathbf{P}). \tag{48}$$

The appendix below summarizes representative graph-handling strategies in a compact form. For clarity, each item isolates the structural design choice rather than a full task-specific architecture.

Cluster and Pooling Methods

Cluster and pooling methods compress the original graph into a smaller set of super-nodes. In differentiable pooling (Ying et al. 2018), the assignment is usually learned as a soft matrix $\mathbf{S} = \mathrm{softmax}(\mathrm{GNN}_{\mathrm{pool}}(\mathbf{A}, \mathbf{H})) \in [0,1]^{N \times M}$, where N is the number of original nodes and M is the number of clusters.

$$\begin{aligned} \mathbf{H}_c &= \mathbf{S}^\top \mathbf{H}, & \mathbf{A}_c &= \mathbf{S}^\top \mathbf{A} \mathbf{S}, \\ \mathbf{H}_c &= \mathbf{D}_c^{-1} \mathbf{C}^\top \mathbf{H}, & \mathbf{A}_c &= \mathbf{C}^\top \mathbf{A} \mathbf{C}, \end{aligned} \tag{49}$$

where $\mathbf{C} \in \{0,1\}^{N \times M}$ is the hard-clustering version of $\mathbf{S}$ and $\mathbf{D}_c = \mathrm{diag}(\mathbf{C}^\top \mathbf{1})$. This is useful when the graph contains stable regions or communities, but the coarsening itself introduces an additional modeling assumption.

Spectral and Graph-Frequency Methods

Spectral methods represent graph signals through the eigenbasis of the graph Laplacian (Defferrard, Bresson, and Vandergheynst 2016). With degree matrix $\mathbf{D}$ and normalized Laplacian $\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$, let $\mathbf{L} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$.

$$\begin{aligned} \mathbf{H}_{\le q} &= \mathbf{V}_q \mathbf{V}_q^\top \mathbf{H}, \\ \mathbf{H}_{>q} &= \left(\mathbf{I} - \mathbf{V}_q \mathbf{V}_q^\top\right) \mathbf{H}. \end{aligned} \tag{50}$$

Low-frequency components capture smooth global trends, whereas high-frequency components emphasize localized deviations and sharp spatial changes, so the coarse-to-fine structure is expressed in the signal domain rather than by removing nodes from the graph.

Adaptive and Dynamic Graphs

Adaptive graph methods learn the connectivity from data instead of fixing it a priori, which is common in spatio-temporal forecasting (Wu et al. 2019). Let $\mathbf{E}_1$ and $\mathbf{E}_2$ be learnable node embeddings:

$$\begin{aligned} \mathbf{A}_\theta &= \mathrm{softmax}_{\mathrm{row}}(\mathbf{E}_1 \mathbf{E}_2^\top), \\ \mathbf{H}' &= \mathbf{A}_\theta \mathbf{H} \mathbf{W}. \end{aligned} \tag{51}$$

The intuition is that hidden statistical correlations need not be fully captured by a fixed physical graph.

Graph Transformers

Graph Transformers combine global attention with graph-aware structural biases (Ying et al. 2021). A representative attention score is

$$\mathrm{Attn}_{ij} = \mathrm{softmax}_j\left(\frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d}} + b_{ij}^G\right). \tag{52}$$

where b_{ij}^G may encode shortest-path distance, edge type, or relative position. The key idea is not simply full connectivity, but global interaction shaped by graph structure.

Positional and Structural Encodings

Positional and structural encodings provide each node with an explicit descriptor of location or structural role (Rampášek et al. 2022). A generic initialization is

$$\begin{aligned} \mathbf{h}_i^0 &= \mathbf{x}_i + \mathbf{p}_i, \\ \mathbf{p}_i &= \left[\mathrm{LapPE}_i, \mathrm{RWSE}_i, \mathrm{SPD}_i\right]^\top. \end{aligned} \tag{53}$$

Such encodings are especially important for graph Transformers, where topology must be reflected through features or attention biases.

Graph Rewiring

Graph rewiring modifies the topology to improve information propagation and reduce bottlenecks (Bober et al. 2022). A compact description is

$$\begin{aligned} \widetilde{\mathbf{A}} &= \mathbf{A} + \Delta\mathbf{A}, \\ \mathbf{H}^{(\ell+1)} &= \mathrm{MPNN}(\mathbf{H}^{(\ell)}, \widetilde{\mathbf{A}}). \end{aligned} \tag{54}$$

The practical goal is to create shorter or more informative paths without discarding the original graph semantics entirely.

Virtual Nodes and Global Tokens

Virtual-node methods insert a global token that aggregates and redistributes information, which is also used in scalable graph Transformers (Shirzad et al. 2023). Let $\mathbf{g}^{(\ell)}$ denote the global state:

$$\begin{aligned} \mathbf{g}^{(\ell+1)} &= \phi_g\left(\mathbf{g}^{(\ell)}, \sum_{i=1}^{N} \mathbf{h}_i^{(\ell)}\right), \\ \mathbf{h}_i^{(\ell+1)} &= \phi_h\left(\mathbf{h}_i^{(\ell)}, \mathbf{g}^{(\ell+1)}\right). \end{aligned} \tag{55}$$

This gives distant nodes a short communication route without requiring many graph layers.

Subgraph, Patch, and Motif Tokens

Subgraph-token methods represent local graph regions as higher-level tokens rather than only as individual nodes (Gao et al. 2022). For a collection of subgraphs $\{S_m\}_{m=1}^{M}$,

$$\begin{aligned} \mathbf{z}_m &= \mathrm{READOUT}\left(\{\mathbf{h}_i : i \in S_m\}\right), \\ \mathbf{Z} &= [\mathbf{z}_1, \ldots, \mathbf{z}_M]. \end{aligned} \tag{56}$$

This is useful when mesoscale patterns or recurring motifs are more stable than nodewise activations, and the selected subgraphs may overlap without replacing the original node-level states.

Hypergraph and Higher-Order Methods

Hypergraph methods replace pairwise edges with multi-node relations. Let $\mathbf{B} \in \{0,1\}^{N \times E}$ be the incidence matrix, where $B_{i,e} = 1$ indicates that node i belongs to hyperedge e. A typical propagation follows a node–hyperedge–node pattern:

$$\mathbf{H}' = \sigma\left(\mathbf{D}_v^{-1}\mathbf{B}\mathbf{W}_e\mathbf{D}_e^{-1}\mathbf{B}^\top\mathbf{H}\mathbf{W}\right). \tag{57}$$

This is appropriate when the relevant interaction is group-based rather than purely pairwise, because $\mathbf{B}^\top\mathbf{H}$ aggregates node features into hyperedge messages before redistribution.

Heterophily, Signed, and Directed Graphs

These graph classes relax the assumption that connected nodes are symmetric and similar (Zheng et al. 2022). A compact operator is

$$\begin{aligned} \mathbf{H}' = \mathbf{A}^{+}\mathbf{H}\mathbf{W}^{+} - \mathbf{A}^{-}\mathbf{H}\mathbf{W}^{-} \\ + \mathbf{A}^{\mathrm{out}}\mathbf{H}\mathbf{W}^{\mathrm{out}} + \mathbf{A}^{\mathrm{in}}\mathbf{H}\mathbf{W}^{\mathrm{in}}. \end{aligned} \tag{58}$$

This perspective is relevant when direction, polarity, or label inconsistency carries task-relevant structure.

Neural Sheaf Methods

Neural sheaf methods replace scalar edge weights with learned linear maps between local feature spaces (Bodnar et al. 2022). A generic update can be written as

$$\mathbf{h}_i' = \sum_{j \in \mathcal{N}(i)} A_{ij}\boldsymbol{\rho}_{ij}\mathbf{h}_j. \tag{59}$$

where $\boldsymbol{\rho}_{ij}$ is an edge-wise linear transport map. This is expressive for heterogeneous neighborhoods, though usually more expensive than standard message passing.

Graph Foundation Models

Graph foundation models pretrain transferable representations and then adapt them to downstream tasks (Liu et al. 2023). A simple abstraction is

$$\begin{aligned} \mathbf{Z} &= \mathrm{Pretrain}_\Theta(\mathcal{G}, \mathbf{X}), \\ f_\theta &= \mathrm{Adapt}(\mathbf{Z}, \mathcal{T}). \end{aligned} \tag{60}$$

The emphasis shifts from fitting a single graph to learning structural representations that transfer across graphs and tasks.

Relay Nodes and RANGE-Style Global Relay

Relay-node methods extend the virtual-node idea by maintaining multiple global relay states (Caruso et al. 2025). Let $\mathbf{R}$ denote the relay matrix:

$$\begin{aligned} \mathbf{R}' &= \mathrm{SA}(\mathbf{R}, \mathbf{H}, \mathbf{P}), \\ \mathbf{H}' &= \mathrm{MPNN}(\mathbf{H}, \mathbf{A}, \mathbf{R}'). \end{aligned} \tag{61}$$

Multiple relay nodes provide higher global communication bandwidth while keeping the propagation pattern sparse and scalable.

Universal Graph Structural Encoders

Universal structural encoders learn transferable structural coordinates once and reuse them across downstream models (Chen et al. 2025). A compact form is

$$\begin{aligned} \mathbf{p}_i &= \mathrm{GFSE}_\Theta(\mathcal{G})_i, \\ \mathbf{h}_i^0 &= \mathbf{x}_i + \mathbf{p}_i. \end{aligned} \tag{62}$$

Compared with hand-crafted encodings, the goal is to amortize structural discovery across tasks and graph domains.

Federated and Multimodal Graph Learning

Federated and multimodal graph learning studies distributed clients, heterogeneous graph sources, and partially observed modalities (Fu et al. 2022; Peng, He, and Xia 2024). A generic objective is

$$\min_\theta \sum_{c=1}^{C} w_c \mathcal{L}_c\left(\theta; \mathcal{G}_c, \mathbf{X}_c^{(m)}\right). \tag{63}$$

The main difficulty is to balance privacy, modality mismatch, and distribution shift across local graph populations.

Relation to Forecast-State Graph Resolution

The proposed method is not graph pooling by itself. Each stage $\omega_s = (h_s, K_s)$ defines a prediction space $\mathcal{Y}_{\omega_s}$, and the full forecast is updated through

$$\widehat{\mathbf{Y}}^{(s)} = \widehat{\mathbf{Y}}^{(s-1)} + \alpha_s \Lambda_s(\mathbf{R}_s). \tag{64}$$

Unlike hard cluster pooling used to replace the original graph with a single coarse graph, the present graph-resolution design repeatedly projects the current forecast with Π_s, predicts a residual in observable prediction coordinates, and lifts that residual back with Λ_s. Unlike latent pooling, the projected object remains an explicit forecast tensor in a lower-resolution target space rather than a generic hidden feature. The distance-aware affinity and capacity constraints therefore serve to define recoverable graph-resolution forecast spaces rather than a terminal coarsened representation.